Administration de bases de données

Informatique — Licence 3 (L3)

Cours INF-L3-14

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Exploiter un SGBD en production comme un administrateur : installation et configuration d'un serveur, stratégie de sauvegarde et de restauration testée, réplication et haute disponibilité, réglage des performances par les index et les plans d'exécution, sécurité des accès au moindre privilège, automatisation et supervision quotidienne.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Installation et configuration d'un SGBD serveur
• Leçon 2 — Sauvegarde et restauration : construire une stratégie complète
• Leçon 3 — Réplication et haute disponibilité des données
• Leçon 4 — Réglage des performances : index et plans d'exécution
• Leçon 5 — Sécurité des bases : rôles, chiffrement et audit des accès
• Leçon 6 — Automatisation des tâches d'administration
• Leçon 7 — Supervision quotidienne et gestion des incidents
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Savoir écrire des requêtes SQL ne suffit pas à faire vivre une base de données. En production, quelqu'un doit installer le serveur, décider qui a le droit de s'y connecter, garantir que les données survivront à une panne de disque, et répondre au téléphone quand l'application devient lente un lundi matin. Ce quelqu'un, c'est l'administrateur de bases de données — le DBA. Ce cours prolonge les cours de SQL et de bases de données avancées (INF-L2-04 et INF-L2-05) et le cours de cloud (INF-L3-05) : on ne conçoit plus le modèle, on exploite le système, jour après jour, avec ses incidents réels. Et quand l'administrateur est absent ou injoignable, c'est toute l'institution qui attend, car étudiants, enseignants et services administratifs dépendent de ces données à chaque instant de la journée. Administrer une base de données est donc un métier de rigueur, de méthode et d'anticipation, qui se prépare longtemps avant la première panne venue, et jamais pendant.

Le fil rouge du cours est la base « biblio » d'une bibliothèque universitaire, hébergée sur un serveur PostgreSQL : quelques milliers de livres, des membres, des emprunts. Petite en volume, elle concentre pourtant tous les problèmes des grandes bases : des sauvegardes qu'il faut pouvoir restaurer, des recherches qui ralentissent quand le catalogue grossit, des comptes à créer sans donner trop de droits, et une exigence simple à formuler, difficile à tenir : ne jamais perdre les données, et ne jamais rester arrêté longtemps. Ses utilisateurs, eux, ne voient jamais le serveur : ils voient seulement des délais qui s'allongent, des erreurs qui surgissent ou un service arrêté — et c'est toujours vers l'administrateur que remontent les questions. Étudier ce cas

jusqu'au bout, c'est apprendre à raisonner comme un DBA : prévoir les pannes au lieu de les découvrir, chiffrer les engagements pris envers les utilisateurs au lieu de les promettre au hasard, et documenter chaque décision pour que l'exploitation survive aux absences comme aux changements d'équipe.

Chaque leçon suit le même rythme : comprendre le concept, voir un exemple réel de commandes ou de scripts (tous les mots de passe sont fictifs et signalés comme tels), puis pratiquer. Deux notions venues du cours INF-L3-05 seront réutilisées avec rigueur : le RPO, quantité maximale de données que l'on accepte de perdre, exprimée en temps, et le RTO, durée maximale d'interruption acceptable avant le retour du service. À la fin du cours, tu sauras dimensionner une exploitation complète : installation, sauvegardes testées, réplication, performances, sécurité, automatisation et supervision. Nous travaillerons principalement avec PostgreSQL, sans perdre de vue les équivalents MySQL : les concepts sont identiques d'un système à l'autre, seuls changent les noms des commandes et des fichiers de configuration. Les travaux dirigés, les exercices et le travail pratique final te feront enfin franchir le pas décisif : passer de la théorie à la mise en exploitation réelle d'une base, jusqu'à simuler un désastre complet et le réparer, chronomètre en main.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Installer un SGBD serveur (PostgreSQL/MySQL) et régler ses paramètres essentiels : mémoire, connexions, journalisation, accès réseau.
- Construire une stratégie de sauvegarde complète (complète, différentielle, incrémentale, journaux) avec des rétentions cohérentes et un RPO/RTO justifiés.
- Restaurer une base à partir de sauvegardes et vérifier que la restauration fonctionne réellement, avant le jour de la panne.
- Expliquer les architectures de réplication (primaire/secondaire, synchrone/asynchrone) et les mécanismes de bascule pour la haute disponibilité.
- Lire un plan d'exécution (EXPLAIN), repérer un balayage séquentiel coûteux et créer l'index qui corrige le problème.
- Sécuriser une base par des rôles au moindre privilège, le chiffrement des échanges et l'audit des accès.
- Automatiser les tâches d'administration (sauvegardes, contrôles, purges) et superviser la base au quotidien.

Leçon 1 — Installation et configuration d'un SGBD serveur

Installer un SGBD serveur ne se résume pas à lancer un assistant et à cliquer sur « Suivant ». Il faut d'abord choisir l'emplacement des données : un disque ou une partition dédiés, séparés du système, pour que la croissance de la base n'étouffe jamais l'exploitation. Il faut ensuite fixer le jeu de caractères (UTF-8, indispensable pour le français et ses accents), le fuseau horaire de référence, et le mode d'écoute réseau : par défaut, un serveur bien installé n'écoute que localement, et l'on n'ouvre l'accès réseau qu'aux machines qui en ont besoin, via le fichier de contrôle d'accès

(pg_hba.conf sous PostgreSQL, comptes hôtes sous MySQL).

La configuration se règle ensuite dans le fichier principal (postgresql.conf ou my.cnf). Trois familles de paramètres dominent : la mémoire (la zone de cache partagée, shared_buffers, réglée en première approximation autour de 25 % de la RAM ; la mémoire de travail par opération, work_mem), le nombre maximal de connexions simultanées (max_connections), et la journalisation des transactions (le WAL, Write-Ahead Log, qui enregistre chaque modification avant son application et rend possibles la reprise après crash et la restauration à un instant donné). Modifier un paramètre sans comprendre son unité ni son effet est le meilleur moyen de dégrader un serveur qui fonctionnait.

Une installation sérieuse se termine par une phase d'hygiène : changer le mot de passe du superutilisateur créé à l'installation, créer un compte d'administration nominatif, désactiver les accès anonymes ou par défaut, redémarrer le service et prouver qu'une connexion réelle fonctionne. On consigne enfin tout cela dans un dossier d'exploitation : version exacte du SGBD, emplacements des fichiers, paramètres modifiés et pourquoi. Le jour où un collègue reprend le serveur, ce document vaut plus que tous les souvenirs de l'administrateur parti en congé.

L'essentiel
Installer un SGBD serveur = choisir l'emplacement des données, fixer encodage et accès réseau au plus juste, régler mémoire/connexions/journalisation en comprenant chaque paramètre, puis sécuriser les comptes par défaut et documenter.

Exemple — Installation et première configuration de PostgreSQL sous Debian

```
# 1) Installation du paquet serveur (version 16)
sudo apt update
sudo apt install -y postgresql-16

# 2) Démarrage automatique du service
sudo systemctl enable --now postgresql

# 3) Première connexion avec le compte système postgres
sudo -u postgres psql -c "SELECT version();"

# 4) postgresql.conf : réglages de départ (serveur 4 Go de RAM)
# shared_buffers = 1GB (environ 25 % de la RAM)
# work_mem = 16MB (par opération de tri/jointure)
# max_connections = 100
# listen_addresses = 'localhost' (réseau à ouvrir au cas par cas)

# 5) pg_hba.conf : la base biblio n'accepte que le réseau local
# de l'université, avec mot de passe chiffré (scram-sha-256)
# host biblio app_biblio 192.168.10.0/24 scram-sha-256

# 6) Prise en compte des réglages sans couper le service
sudo -u postgres psql -c "SELECT pg_reload_conf();"

# 7) Mot de passe du superutilisateur (valeur fictive, à remplacer)
sudo -u postgres psql -c "ALTER ROLE postgres PASSWORD 'Fictif_Admin_2026!';"
```

À retenir

• Séparer les données du système et fixer UTF-8 dès l'installation : ces deux choix sont presque impossibles à corriger plus tard.

• shared_buffers, work_mem et max_connections se dimensionnent ensemble : 100 connexions × work_mem élevé peut épuiser toute la RAM.

• Le contrôle d'accès (pg_hba.conf) décide qui se connecte, depuis quelle machine et avec quelle preuve : c'est le premier pare-feu de la base.

Exercice

Un serveur dispose de 8 Go de RAM et doit accueillir PostgreSQL avec au maximum 200 connexions simultanées. Un collègue propose : shared_buffers = 6 Go et work_mem = 128 Mo. Explique pourquoi cette configuration est dangereuse, puis propose des valeurs plus raisonnables en justifiant chaque choix.

✔ Corrigé
Le piège est dans work_mem : ce paramètre n'est pas global, il vaut pour chaque opération de tri ou de jointure, et une requête complexe peut en déclencher plusieurs. Avec 200 connexions et work_mem = 128 Mo, le pire cas dépasse très largement 8 Go : le serveur échangerait sur disque (swap) voire serait tué par le système. De plus, shared_buffers = 6 Go ne laisse que 2 Go au système d'exploitation, à son cache disque et aux processus eux-mêmes. Proposition raisonnable : shared_buffers = 2 Go (25 % de 8 Go), work_mem = 16 Mo (200 connexions resteraient sous contrôle, quitte à augmenter ponctuellement pour une session d'administration via SET work_mem), max_connections = 200 conservé ou réduit en plaçant un pooler de connexions (type PgBouncer) devant l'application. La règle : faire l'arithmétique du pire cas avant d'écrire le paramètre.

Leçon 2 — Sauvegarde et restauration : construire une stratégie complète

Trois types de sauvegardes structurent toutes les stratégies, et il faut cesser de les confondre. La sauvegarde complète copie l'ensemble des données à un instant donné : c'est la référence à partir de laquelle tout se reconstruit. La sauvegarde différentielle copie uniquement ce qui a changé depuis la dernière sauvegarde COMPLÈTE : elle grossit donc de jour en jour, jusqu'à la prochaine complète. La sauvegarde incrémentale copie ce qui a changé depuis la dernière sauvegarde, qu'elle soit complète ou incrémentale : chaque fichier est petit, mais restaurer suppose de rejouer toute la chaîne, dans l'ordre, sans un seul maillon manquant.

La conséquence pratique est immédiate : pour restaurer une différentielle, il faut deux éléments — la complète et la dernière différentielle. Pour restaurer une chaîne incrémentale de six jours, il en faut sept — la complète et les six incrémentales, dans l'ordre chronologique. On ajoute souvent un quatrième élément : les journaux de transactions (WAL sous PostgreSQL, binary logs sous MySQL), archivés en continu. En les rejouant après la restauration d'une complète, on obtient la restauration à un instant précis (PITR, Point-In-Time Recovery) : revenir à 10 h 04, juste avant la fausse manipulation de 10 h 05.

Une stratégie se dimensionne à partir de deux engagements, définis dans le cours INF-L3-05 : le RPO (perte de données maximale acceptable, en temps) décide de la fréquence des sauvegardes et de l'archivage des journaux ; le RTO (durée maximale d'interruption) décide des moyens de restauration. Les rétentions disent combien de temps on garde chaque génération : par exemple, complètes gardées 4 semaines, différentielles 7 jours, journaux 7 jours. La règle 3-2-1 complète le tableau : 3 copies, sur 2 supports différents, dont 1 hors site. Et la règle d'or, qui vaut tous les certificats : une sauvegarde jamais testée en restauration n'existe pas.

L'essentiel
Complète = tout, référence. Différentielle = changements depuis la dernière complète (restauration : complète + 1 différentielle). Incrémentale = changements depuis la dernière sauvegarde (restauration : la chaîne entière). Journaux archivés = restauration à un instant précis. RPO fixe les fréquences, RTO fixe les moyens, et on teste les restaurations.

Exemple — Plan de sauvegarde complet de la base biblio (PostgreSQL)

```
# Objectif : RPO = 15 minutes, RTO = 2 heures
# ------------------------------------------------------------
# SAUVEGARDE COMPLÈTE logique, chaque dimanche à 2 h 00
# Rétention : 4 générations hebdomadaires
pg_dump -U postgres -Fc biblio -f /sauvegardes/biblio_complete_dim.dump

# ÉQUIVALENT MySQL (sauvegarde complète cohérente)
mysqldump -u root -p --single-transaction --routines biblio > biblio_complete_dim.sql

# SAUVEGARDES DIFFÉRENTIELLES (par rapport à la complète du dimanche)
# -> en pratique sous PostgreSQL : sauvegarde physique de base
# le lundi, puis archivage continu des journaux (WAL) ;
# sous SQL Server : BACKUP ... WITH DIFFERENTIAL.
# Rétention : 7 jours (une par jour, lundi -> samedi)

# JOURNAUX DE TRANSACTIONS : archivés en continu vers un 2e support
# postgresql.conf :
# archive_mode = on
# archive_command = 'test ! -f /archives_wal/%f && cp %p /archives_wal/%f'
# => perte maximale = 15 min de transactions (le RPO visé)

# RESTAURATION à un instant précis (PITR), par exemple 10 h 04 :
# 1) restaurer la complète (ou la base physique) du dimanche
# 2) rejouer les WAL jusqu'à l'heure cible :
# recovery_target_time = '2026-10-11 10:04:00'
# COPIE HORS SITE : les fichiers partent chaque nuit vers un
# stockage distant (règle 3-2-1 : 3 copies, 2 supports, 1 hors site)
```

À retenir

- Différentielle se réfère toujours à la dernière complète ; incrémentale se réfère toujours à la dernière sauvegarde, quelle qu'elle soit.
- Restaurer une différentielle demande 2 fichiers ; restaurer une chaîne incrémentale demande toute la chaîne, dans l'ordre.
- RPO et RTO ne sont pas des vœux : ils se déduisent du plan (fréquence, journaux, support) et se prouvent par des tests de restauration chronométrés.

Exercice

Le plan d'une PME est le suivant : complète le dimanche soir, incrémentales du lundi au samedi soir. Le disque tombe en panne le vendredi à 15 h. 1) Quels fichiers faut-il rassembler pour restaurer l'état du jeudi soir, et dans quel ordre les appliquer ? 2)

Même question si les sauvegardes de semaine étaient des différentielles. 3) Quelles données sont perdues dans les deux cas, si aucun journal de transactions n'est archivé ?

✔ Corrigé
1) Incrémentales : il faut la complète du dimanche, puis les incrémentales du lundi, mardi, mercredi et jeudi, appliquées dans cet ordre chronologique strict. Cinq fichiers, un seul manquant ou corrompu rend toute la fin de la chaîne inutilisable. 2) Différentielles : deux fichiers suffisent — la complète du dimanche et la différentielle du jeudi soir, qui contient à elle seule tous les changements depuis dimanche. 3) Sans journaux archivés, le dernier état récupérable est celui du jeudi soir dans les deux cas : toutes les transactions du vendredi (de 0 h au crash de 15 h) sont perdues. Le RPO réel de ce plan est donc « jusqu'à 24 h de données », quelle que soit la famille choisie — seule l'archivage des journaux permet de descendre vers quelques minutes.

Leçon 3 — Réplication et haute disponibilité des données

La réplication consiste à maintenir une ou plusieurs copies d'une base sur d'autres serveurs, alimentées en continu par les modifications du serveur principal — le primaire (ou maître) — vers ses secondaires (ou répliques). Elle répond à deux besoins différents qu'il ne faut pas mélanger : répartir la charge de lecture (les rapports et statistiques lisent sur un secondaire sans gêner les écritures) et surtout survivre à la mort du primaire : c'est la haute disponibilité. Sous PostgreSQL, la réplication dite « en flux » (streaming) expédie les journaux WAL au fur et à mesure ; sous MySQL, ce sont les binary logs et les GTID qui jouent ce rôle.

Le choix structurant est le mode de validation. En réplication asynchrone, le primaire valide la transaction sans attendre le secondaire : c'est rapide, mais si le primaire meurt brutalement, les dernières transactions non encore transmises sont perdues — le RPO n'est pas nul, il est seulement petit et non garanti. En réplication synchrone, le primaire attend la confirmation d'au moins un secondaire avant de valider : aucune transaction confirmée n'est perdue, au prix d'une latence d'écriture plus élevée et d'un blocage si le secondaire tombe. Ce compromis — perte possible contre lenteur — se décide avec le métier, jamais par habitude.

Disposer d'un secondaire ne suffit pas : il faut la bascule (failover), c'est-à-dire promouvoir le secondaire en nouveau primaire et rediriger les applications vers lui. Elle peut être manuelle (un administrateur vérifie puis promeut, en quelques minutes) ou automatisée par des outils de supervision (Patroni, repmgr, MySQL InnoDB Cluster), qui détectent la panne et basculent en quelques dizaines de secondes. Deux pièges guettent : le retard de réplication, qui fait lire des données périmées sur un secondaire asynchrone, et le « split-brain », où deux serveurs se croient primaires en même temps et divergent définitivement. Dernier point, martelé : une réplique n'est pas une sauvegarde — une suppression de table est répliquée fidèlement en quelques millisecondes.

L'essentiel
La réplication copie en continu le primaire vers des secondaires (WAL chez PostgreSQL, binary logs chez MySQL). Asynchrone : rapide, perte possible des dernières transactions. Synchrone : aucune perte confirmée, écriture plus lente. La bascule promeut un secondaire en cas de panne. Et une réplique ne remplace jamais une sauvegarde.

Exemple — Réplication en flux PostgreSQL : primaire et secondaire

```
# ===== Sur le PRIMAIRE (192.168.10.21) =====
# postgresql.conf :
# wal_level = replica
# max_wal_senders = 5
# archive_mode = on
# Rôle dédié à la réplication (mot de passe fictif) :
CREATE ROLE repl LOGIN REPLICATION PASSWORD 'Fictif_Repl_2026!';
-- pg_hba.conf :
-- host replication repl 192.168.10.22/32 scram-sha-256

# ===== Sur le SECONDAIRE (192.168.10.22), serveur arrêté =====
# Copie physique initiale du primaire (-R écrit la config de standby)
pg_basebackup -h 192.168.10.21 -U repl -D /var/lib/postgresql/16/main -R -P
# Puis démarrer PostgreSQL : il se met en récupération et suit le flux

# ===== Contrôle, côté primaire =====
SELECT client_addr, state, sync_state, replay_lag
FROM pg_stat_replication;
-- state = streaming : le secondaire reçoit le flux en direct
-- replay_lag : son retard d'application (à surveiller !)

# ===== Bascule planifiée (maintenance du primaire) =====
# Sur le secondaire, après arrêt propre du primaire :
pg_ctl promote -D /var/lib/postgresql/16/main
-- Le secondaire devient primaire ; on reconfigure ensuite
-- l'ancien primaire en secondaire du nouveau (ou on le reconstruit)
```

À retenir

- Asynchrone ou synchrone : on choisit entre vitesse d'écriture et garantie de zéro perte — et on documente le RPO qui en découle.
- Le retard de réplication (replay_lag) fait partie de la supervision quotidienne : un secondaire en retard est une fausse sécurité.
- Réplication ≠ sauvegarde : les erreurs logiques (DELETE, DROP) se répliquent instantanément ; seuls les backups protègent contre elles.

Exercice

Une banque exige qu'aucune transaction confirmée à un client ne puisse être perdue, même si le serveur primaire brûle. Une boutique en ligne préfère, elle, des écritures rapides et accepte de perdre au pire quelques secondes de commandes en cas de crash. Pour chaque cas, choisis le mode de réplication et justifie. Puis explique

pourquoi, même dans le cas de la banque, la réplication ne dispense pas de sauvegardes quotidiennes.

✔ Corrigé
Banque : réplication synchrone (au moins un secondaire confirme avant le COMMIT) — le primaire ne confirme au client qu'après réception par le secondaire, donc une transaction confirmée existe sur deux machines ; le prix est une latence d'écriture accrue et l'exigence d'un secondaire toujours joignable. Boutique : réplication asynchrone — écritures au plus vite, RPO de quelques secondes assumé contractuellement. Dans les deux cas, les sauvegardes restent indispensables : la réplication propage les données, y compris les erreurs et les corruptions logiques (un DELETE massif, une table supprimée, un rançongiciel qui chiffre les fichiers). Seule une sauvegarde antérieure, isolée et testée permet de revenir en arrière dans le temps ; la réplication ne sait faire qu'une chose : copier le présent.

Leçon 4 — Réglage des performances : index et plans d'exécution

Quand une requête devient lente, on ne devine pas : on lit son plan d'exécution. La commande EXPLAIN demande au SGBD d'expliquer comment il compte s'y prendre : quelles tables, dans quel ordre, par quelle méthode d'accès. Le mot qui doit alerter est le balayage séquentiel (Seq Scan sous PostgreSQL, SCAN sous SQLite, type ALL sous MySQL) : le moteur lit la table entière, ligne par ligne, pour n'en garder que quelques-unes. Sur une table de dix lignes, c'est anodin ; sur un million de lignes consultées cent fois par minute, c'est le début de la panne de production.

L'index est la réponse classique à ce problème : une structure triée (un arbre B, le plus souvent) qui associe chaque valeur d'une colonne à l'emplacement des lignes correspondantes — exactement l'index d'un livre, qui évite de relire toutes les pages pour trouver un mot. Encore faut-il le créer au bon endroit : colonnes des clauses WHERE fréquentes, colonnes de jointure (les clés étrangères ne sont pas indexées automatiquement partout !), colonnes de tri. L'exemple ci-dessous est réel : il a été exécuté sous SQLite pour ce cours, et l'on y voit le plan passer de SCAN (lecture complète) à SEARCH USING INDEX dès l'index créé.

L'index n'est pourtant pas gratuit : il occupe de l'espace disque, et chaque écriture (INSERT, UPDATE, DELETE) doit le mettre à jour. Indexer toutes les colonnes « au cas où » transforme une base rapide en lecture et paresseuse en écriture. La méthode professionnelle est donc : mesurer (journal des requêtes lentes), expliquer (EXPLAIN, puis EXPLAIN ANALYZE qui exécute réellement et chronomètre), indexer avec parcimonie, re-mesurer, et tenir les statistiques du moteur à jour (ANALYZE, autovacuum) pour qu'il continue de choisir de bons plans au fil de la croissance des tables.

L'essentiel
Le plan d'exécution (EXPLAIN) montre comment la requête est réellement traitée : un balayage complet (SCAN / Seq Scan) sur une grosse table filtrée est le signal d'un index manquant. L'index accélère les lectures mais coûte de l'espace et ralentit chaque écriture : on indexe ce que les requêtes filtrent et joignent, après mesure.

Exemple — EXPLAIN réel (exécuté sous SQLite) : avant et après index

```
-- Base d'experience : livres(id, titre, auteur, annee)
-- emprunts(id, livre_id, membre, date_emprunt)
-- Requete : les livres d'un auteur donne

-- ===== AVANT la creation d'un index =====
EXPLAIN QUERY PLAN
SELECT titre FROM livres WHERE auteur = 'Victor Hugo';
-- Resultat reel :
-- `--SCAN livres
-- -> lecture COMPLETE de la table, ligne par ligne

-- ===== Creation de l'index =====
CREATE INDEX idx_livres_auteur ON livres(auteur);

-- ===== APRES =====
EXPLAIN QUERY PLAN
SELECT titre FROM livres WHERE auteur = 'Victor Hugo';
-- Resultat reel :
-- `--SEARCH livres USING INDEX idx_livres_auteur (auteur=?)
-- -> acces direct aux lignes concernees, plus de balayage

-- ===== Cas des jointures : indexer la colonne filtree =====
EXPLAIN QUERY PLAN
SELECT l.titre, e.membre FROM emprunts e
JOIN livres l ON l.id = e.livre_id WHERE e.membre = 'Awa';
-- Avant : SCAN e (emprunts lue en entier)
CREATE INDEX idx_emprunts_membre ON emprunts(membre);
-- Apres (reel) :
-- |--SEARCH e USING INDEX idx_emprunts_membre (membre=?)
-- `--SEARCH l USING INTEGER PRIMARY KEY (rowid=?)
```

À retenir

- EXPLAIN d'abord, index ensuite : on ne crée jamais un index sur une intuition, mais sur un plan lu et une lenteur mesurée.
- Candidats à l'index : colonnes de filtres fréquents, colonnes de jointure, colonnes de tri — pas les colonnes rarement interrogées.
- Chaque index ralentit les écritures et consomme du disque : un index jamais utilisé (visible dans les statistiques) doit être supprimé.

Exercice

Le journal des requêtes lentes d'une boutique montre cette requête exécutée 4 000 fois par jour : SELECT * FROM commandes WHERE client_id = 512 AND statut =

'en_attente'. La table commandes compte 800 000 lignes et ne possède qu'un index sur sa clé primaire. 1) Quel plan obtiendrait-on probablement avec EXPLAIN, et pourquoi est-ce grave à ce volume ? 2) Propose l'index adapté et explique l'ordre des colonnes. 3) Cite un cas où ce même index ne servirait à rien.

✔ Corrigé
1) Un balayage séquentiel de 800 000 lignes pour ne retenir que les commandes d'un client dans un statut donné : répété 4 000 fois par jour, il monopolise le disque et le processeur. 2) Un index composite : CREATE INDEX idx_commandes_client_statut ON commandes(client_id, statut). On met d'abord client_id, la colonne la plus sélective (elle divise la table par le nombre de clients), puis statut : l'index permet alors d'atteindre directement le petit intervalle de lignes cherché, et il sert aussi aux requêtes filtrant sur client_id seul. 3) Il ne servirait à rien pour une requête sans filtre sur ces colonnes (par exemple le total des ventes du mois sur toute la table, qui doit de toute façon tout lire), ni pour un filtre portant uniquement sur statut si ce statut concerne la moitié des lignes — le moteur préfère alors, à juste titre, le balayage complet.

Leçon 5 — Sécurité des bases : rôles, chiffrement et audit des accès

La sécurité d'une base repose d'abord sur les rôles : des comptes nommés, chacun porteur de droits précis. Le principe directeur est le moindre privilège : un compte ne reçoit que les droits strictement nécessaires à sa tâche, rien de plus. Concrètement, l'application se connecte avec un rôle applicatif qui peut lire et modifier les tables de son schéma, mais ni créer de base, ni gérer les autres rôles, et surtout pas superutilisateur. À côté vivent un rôle de lecture seule pour les rapports, et des comptes d'administration nominatifs — un par personne, jamais un compte « admin » partagé dont on ne sait plus qui l'a utilisé.

Les droits se règlent finement avec GRANT et REVOKE : par base, par schéma, par table, voire par colonne, et par opération (SELECT, INSERT, UPDATE, DELETE). Quelques réflexes valent de l'or : retirer les droits accordés par défaut au pseudo-rôle PUBLIC, exiger des mots de passe robustes stockés sous forme d'empreinte (scram-sha-256 sous PostgreSQL), limiter les hôtes autorisés à se connecter dans pg_hba.conf, et chiffrer les connexions réseau (ssl = on) dès que le client et le serveur ne sont pas sur la même machine. Les données au repos peuvent aussi être chiffrées au niveau du disque ou du système de fichiers — protection contre le vol physique du serveur, pas contre un compte compromis.

Reste l'audit : savoir qui a fait quoi, et quand. On journalise les connexions et déconnexions, les échecs d'authentification, et selon la sensibilité, les modifications de structure ou les accès aux tables sensibles. Ces journaux ne servent à rien s'ils dorment sur le serveur : on les relit (une alerte sur les échecs répétés détecte une attaque par force brute), on les conserve assez longtemps, et on revoit périodiquement la liste des rôles et de leurs droits — les comptes des personnes parties et les droits accordés « temporairement » il y a deux ans sont des classiques des audits qui font mal.

L'essentiel
Moindre privilège pour chaque rôle : l'application n'est jamais superutilisateur, les rapports sont en lecture seule, l'administration est nominative. GRANT précis, PUBLIC dépouillé, mots de passe à empreinte, connexions chiffrées, et journalisation relue des connexions comme des échecs.

Exemple — Rôles au moindre privilège pour la base biblio (PostgreSQL)

```
-- Mots de passe fictifs, à remplacer par des valeurs fortes
-- 1) Role de l'application : lire et ecrire les tables, rien de plus
CREATE ROLE app_biblio LOGIN PASSWORD 'Fictif_App_2026!';

-- 2) Role de consultation (rapports, statistiques) : lecture seule
CREATE ROLE lecture_biblio LOGIN PASSWORD 'Fictif_Lecture_2026!';

-- 3) Role d'administration nominatif : peut creer des objets,
-- mais PAS superutilisateur (reserve au compte postgres)
CREATE ROLE admin_biblio LOGIN PASSWORD 'Fictif_Admin_2026!'
CREATEDB CREATEROLE;

-- 4) Connexion a la base biblio
GRANT CONNECT ON DATABASE biblio TO app_biblio, lecture_biblio;

-- 5) Acces au schema
GRANT USAGE ON SCHEMA public TO app_biblio, lecture_biblio;

-- 6) Droits sur les tables : ecriture pour l'application...
GRANT SELECT, INSERT, UPDATE, DELETE
ON ALL TABLES IN SCHEMA public TO app_biblio;
-- ...lecture seule pour la consultation
GRANT SELECT ON ALL TABLES IN SCHEMA public TO lecture_biblio;

-- 7) Retirer les droits accordes par defaut a tout le monde
REVOKE ALL ON DATABASE biblio FROM PUBLIC;

-- Verification : \du sous psql liste les roles et leurs attributs.
-- Aucun des trois roles ne doit afficher « Superutilisateur ».
```

À retenir

- Jamais de superutilisateur pour une application : un bug ou une injection SQL avec ce compte, et c'est tout le serveur qui tombe.
- Un rôle par usage (application, lecture, administration nominative) permet de révoquer proprement et de savoir qui a fait quoi.
- L'audit n'existe que s'il est relu : échecs de connexion répétés et comptes dormants sont les deux signaux à traquer.

Exercice

Voici les pratiques relevées lors d'un audit : (a) l'application web se connecte avec le compte superutilisateur postgres ; (b) tous les développeurs partagent un même

compte « dev » avec tous les droits ; (c) le rôle du stagiaire parti il y a six mois est toujours actif ; (d) les connexions se font sans chiffrement depuis un autre bâtiment. Pour chaque point, explique le risque concret et la correction à appliquer.

✔ Corrigé
(a) Risque majeur : une faille applicative (par exemple une injection SQL) hérite des droits du superutilisateur — suppression de bases, création de comptes, lecture de tout le serveur. Correction : créer un rôle applicatif limité aux tables nécessaires (SELECT/INSERT/UPDATE/DELETE sur le schéma métier). (b) Aucune traçabilité : impossible d'attribuer une action à une personne, et le mot de passe partagé finit connu de tous. Correction : un compte nominatif par développeur, droits réduits (pas d'accès à la production si elle existe). (c) Un compte orphelin est une porte d'entrée discrète. Correction : désactiver immédiatement (NOLOGIN) puis supprimer après vérification de ses objets ; mettre en place une revue trimestrielle des rôles. (d) Les mots de passe et les données circulent lisibles sur le réseau. Correction : activer SSL côté serveur (ssl = on, certificats) et exiger hostssl dans pg_hba.conf pour les connexions distantes.

Leçon 6 — Automatisation des tâches d'administration

Tout ce qui doit arriver régulièrement finit, un jour ou l'autre, par être oublié un dimanche soir : l'administrateur professionnel automatise donc tout ce qui est répétitif. Les cibles naturelles sont les sauvegardes (leçon 2), la purge des anciennes générations et des journaux, les contrôles d'intégrité, la mise à jour des statistiques (ANALYZE) et le nettoyage des lignes mortes (VACUUM sous PostgreSQL, OPTIMIZE TABLE sous MySQL, largement délégués aux processus automatiques mais à surveiller). Sous Linux, l'ordonnanceur du système est cron ; sous Windows, le Planificateur de tâches ; PostgreSQL dispose aussi d'extensions comme pg_cron pour planifier au plus près de la base.

Automatiser ne veut pas dire « écrire un script et ne plus jamais le regarder ». Un bon script d'administration respecte quatre règles : il journalise tout ce qu'il fait (quoi, quand, succès ou échec, dans un fichier daté) ; il rend un code de sortie honnête (0 si tout va bien, autre chose sinon) pour que la supervision puisse alerter ; il vérifie ses résultats (taille du fichier de sauvegarde non nulle, test de restauration périodique sur une machine de secours) ; et il refuse de détruire en silence — la purge ne supprime que ce qui respecte strictement la rétention définie, jamais « tout le dossier ».

L'automatisation a aussi ses propres pannes : un script qui échoue chaque nuit sans que personne ne lise son journal est pire qu'une tâche manuelle, car il donne une fausse confiance. On surveille donc l'automate lui-même : alerte si la sauvegarde de la nuit n'est pas arrivée, rapport hebdomadaire récapitulatif, et test de restauration automatique programmé — par exemple chaque mois, restaurer la dernière complète sur un serveur de test et y compter les lignes de quelques tables témoins. C'est ce contrôle, plus que la sauvegarde elle-même, qui prouve que le RPO et le RTO annoncés sont réels.

L'essentiel
On automatise sauvegardes, purges, contrôles d'intégrité et statistiques avec cron (ou le Planificateur de tâches), mais chaque script doit journaliser, signaler ses échecs, vérifier ses résultats et être lui-même supervisé — avec un test de restauration automatique comme preuve finale.

Exemple — Tâches cron d'administration de la base biblio (commentées)

```
# Crontab de l'utilisateur postgres : crontab -e
# Format : minute heure jour mois jour-semaine commande
# (jour-semaine : 0 = dimanche, 1 = lundi, ... 6 = samedi)

# 1) Sauvegarde complete le dimanche a 2 h 00
0 2 * * 0 /usr/local/bin/sauvegarde_biblio.sh complete >>
/var/log/sauvegarde_biblio.log 2>&1

# 2) Sauvegarde differentielle du lundi au samedi a 2 h 00
0 2 * * 1-6 /usr/local/bin/sauvegarde_biblio.sh differentielle >>
/var/log/sauvegarde_biblio.log 2>&1

# 3) Purge des sauvegardes de plus de 28 jours, chaque jour a 4 h 30
# (find ne cible QUE les fichiers .dump du dossier dedie)
30 4 * * * find /sauvegardes/biblio -name '*.dump' -mtime +28 -delete

# 4) Controle hebdomadaire le lundi a 5 h 00 : la derniere
# sauvegarde complete est restauree sur le serveur de test,
# et le nombre de livres comptes est compare a la production
0 5 * * 1 /usr/local/bin/controle_restauration.sh >>
/var/log/controle_restauration.log 2>&1

# Le script de sauvegarde doit : dater son fichier, verifier
# que sa taille est > 0, et quitter avec un code non nul en
# cas d'echec pour declencher l'alerte de la supervision.
```

À retenir

- Cron planifie, mais ne surveille pas : sans journalisation ni alerte sur absence de résultat, une automatisation en panne passe inaperçue des mois.
- Une purge automatique se borne à un dossier et un motif de fichiers précis : jamais de suppression large et aveugle.
- Le test de restauration périodique automatisé est la seule preuve que les sauvegardes — et donc le RPO/RTO — sont réels.

Exercice

Écris les lignes cron (avec commentaire) pour : (a) lancer un script de statistiques /usr/local/bin/stats_biblio.sh tous les jours à 3 h 15 ; (b) lancer un nettoyage /usr/local/bin/nettoyage_biblio.sh le premier jour de chaque mois à 1 h 00 ; (c) vérifier l'espace disque toutes les 30 minutes. Puis explique ce qui se passerait si le script de sauvegarde plantait chaque nuit sans journalisation ni code de sortie d'erreur.

✔ Corrigé
(a) 15 3 * * * /usr/local/bin/stats_biblio.sh — tous les jours à 3 h 15. (b) 0 1 1 * * /usr/local/bin/nettoyage_biblio.sh — minute 0, heure 1, jour du mois = 1. (c) */30 * * * * suivi de la commande de contrôle d'espace disque — le motif */30 signifie « toutes les 30 minutes ». Sans journalisation ni code de sortie significatif, le plantage nocturne serait invisible : l'équipe croirait disposer de sauvegardes récentes alors que la dernière réussie daterait de semaines. Le jour de l'incident réel, la restauration échouerait et le RPO réel se révélerait catastrophique — c'est exactement le scénario que la supervision de l'automatisation (alerte si aucun fichier de sauvegarde récent) est chargée d'empêcher.

Leçon 7 — Supervision quotidienne et gestion des incidents

Superviser une base, c'est répondre chaque jour à quatre questions simples : le service est-il vivant et joignable ? Les données tiennent-elles encore longtemps sur le disque ? Les utilisateurs sont-ils servis à une vitesse acceptable ? Et les mécanismes de protection (sauvegarde de la nuit, réplication) sont-ils en bonne santé ? Concrètement, on surveille la disponibilité du processus, le nombre de connexions actives, la taille des bases et de leurs plus grosses tables, les requêtes les plus lentes (journal des requêtes lentes, pg_stat_statements sous PostgreSQL), les erreurs du journal du SGBD, le retard de réplication et le résultat des sauvegardes. Un tableau de bord et des seuils d'alerte transforment ces mesures en décisions avant que les utilisateurs ne téléphonent.

La routine du DBA combine du quotidien et du périodique. Chaque matin, un coup d'œil de dix minutes : sauvegardes de la nuit réussies, erreurs nouvelles dans les journaux, espace disque, retard du secondaire. Chaque semaine : les requêtes les plus coûteuses, les index inutilisés ou manquants, la croissance des tables. Chaque mois : test de restauration complet, revue des rôles et des droits, bilan de capacité — dans combien de semaines le disque sera-t-il plein au rythme actuel ? Cette discipline ennuyeuse est précisément ce qui distingue une base « qui n'a jamais de problème » d'une base dont on découvre les problèmes en production, un jour de pointe.

Malgré tout, l'incident arrive : disque plein, corruption détectée, requêtes bloquées en cascade, primaire injoignable. La gestion suit alors une méthode : constater et qualifier (qui est touché ? depuis quand ?), contenir (arrêter l'hémorragie : tuer la requête bloquante, libérer de l'espace, basculer sur le secondaire), restaurer le service, puis seulement diagnostiquer la cause racine et documenter. Deux outils font gagner un temps précieux le jour J : une procédure écrite à l'avance (qui appeler, quelle bascule, quelle restauration) et un post-mortem sans blâme après coup, qui transforme chaque incident en amélioration du plan de sauvegarde, de la supervision ou de la configuration.

L'essentiel
Superviser = disponibilité, espace, lenteurs, santé des sauvegardes et de la réplication, avec des seuils d'alerte. Routine quotidienne, hebdomadaire et mensuelle. En incident : qualifier, contenir, restaurer, puis analyser la cause et documenter — avec des procédures écrites avant la crise.

Exemple — Contrôles SQL de supervision quotidienne (PostgreSQL)

```
-- 1) Taille des bases : anticiper le disque plein
SELECT datname,
pg_size_pretty(pg_database_size(datname)) AS taille
FROM pg_stat_database ORDER BY pg_database_size(datname) DESC;

-- 2) Requetes actives depuis plus de 5 minutes : blocages ?
SELECT pid, usename, state,
now() - query_start AS duree,
left(query, 60) AS debut_requete
FROM pg_stat_activity
WHERE state = 'active' AND now() - query_start > interval '5 minutes';

-- 3) Retard du secondaire (a executer SUR le secondaire)
SELECT now() - pg_last_xact_replay_timestamp()
AS retard_replication;
-- Au-dela du seuil fixe (ex. 5 minutes) : alerte.

-- 4) Index jamais utilises : candidats a la suppression
SELECT relname AS table_concernee, indexrelname AS index_inutile,
idx_scan AS nombre_d_utilisations
FROM pg_stat_user_indexes WHERE idx_scan = 0;

-- 5) Sante des sauvegardes : verifier chaque matin qu'un
-- fichier de la nuit existe et a une taille credible
-- (cote shell : ls -lh /sauvegardes/biblio/ | tail)
```

À retenir

- Les quatre santés à contrôler : service, espace, vitesse, protections (sauvegarde + réplication) — une seule en rouge suffit à agir.
- Les seuils transforment des chiffres en alertes : retard de réplication, disque à 80 %, requête de plus de 5 minutes.
- En incident, on restaure le service avant de chercher le coupable ; l'analyse de cause racine vient après, et elle se documente.

Exercice

Scénario d'incident : un lundi à 9 h 05, les utilisateurs signalent que l'application de la bibliothèque « ne répond plus ». Tu constates que le disque du serveur de base est plein à 100 %. Décris ta réponse dans l'ordre : actions immédiates pour contenir, rétablissement du service, puis mesures pour que cela ne se reproduise pas. Précise quelle question de supervision quotidienne aurait dû alerter dès la semaine précédente.

✔ Corrigé
Contenir : identifier ce qui a rempli le disque (journaux applicatifs, anciennes sauvegardes jamais purgées, fichiers temporaires) et libérer de l'espace de façon sûre — purger les fichiers visiblement obsolètes et hors données, jamais supprimer des fichiers de la base elle-même (un fichier WAL ou de données effacé « pour faire de la place » transforme un incident en catastrophe). Vérifier ensuite que PostgreSQL accepte de nouveau les écritures et que la réplication repart. Rétablir : contrôler l'intégrité (la base n'a pas été corrompue par l'arrêt en pleine écriture), confirmer aux utilisateurs le retour du service, chiffrer la durée d'indisponibilité réelle face au RTO engagé. Prévenir : l'alerte manquée était la croissance du disque — la supervision quotidienne de la taille des bases et de l'espace libre, avec un seuil (80 %) et une projection de tendance (« plein dans 12 jours au rythme actuel »), aurait déclenché l'action la semaine précédente : extension du volume, correction de la purge des sauvegardes, ou archivage des anciennes données. Le post-mortem consigne la cause racine et la nouvelle alerte mise en place.

Travaux dirigés (TD)

TD 1 — Concevoir la stratégie de sauvegarde d'une PME

Une PME de distribution emploie 60 personnes autour d'une base PostgreSQL de gestion commerciale (commandes, stock, facturation) de 40 Go, modifiée toute la journée. La direction fixe : RPO = 1 heure (on accepte de perdre au plus une heure de saisie), RTO = 4 heures (le service doit revenir dans la demi-journée). Le budget matériel est limité : un serveur principal, un second serveur plus modeste, et un stockage distant. Propose une stratégie complète : types de sauvegardes et fréquences, place des journaux de transactions, rétentions, rôle du second serveur, et procédure de test. Justifie chaque choix par le RPO ou le RTO.

✔ Corrigé du TD 1
RPO = 1 h impose de ne jamais dépendre d'une sauvegarde quotidienne : on met en place une sauvegarde complète physique (pg_basebackup) chaque nuit, plus l'archivage continu des journaux WAL vers le second serveur et le stockage distant. La perte maximale devient l'intervalle d'archivage : en archivant chaque segment WAL dès sa clôture (et au plus tard toutes les 15 minutes via archive_timeout), le RPO réel tombe sous l'heure exigée, avec marge. RTO = 4 h : le second serveur héberge un secondaire en réplication en flux asynchrone (alimenté par ces mêmes WAL) ; en cas de panne du primaire, la bascule par promotion du secondaire ramène le service en quelques dizaines de minutes, très en dessous des 4 h — l'alternative « restaurer 40 Go depuis la complète » resterait possible mais plus lente et réservée à la corruption logique. Rétentions : complètes 14 jours, WAL 14 jours, une complète mensuelle conservée 12 mois pour les besoins comptables. Copies : règle 3-2-1 (production, second serveur, stockage distant). Tests : restauration complète chronométrée chaque mois sur le second serveur (hors bascule), avec vérification de quelques totaux de facturation ; le temps mesuré est consigné et comparé au RTO. Enfin, la restauration à un instant précis (PITR) couvre le cas de l'erreur humaine — la fausse manipulation — que la réplication, elle, recopierait instantanément.

TD 2 — Diagnostiquer une base devenue lente

Le catalogue en ligne d'une médiathèque mettait 0,2 seconde à répondre ; depuis que la table des ouvrages est passée de 20 000 à 900 000 lignes, la recherche par auteur prend 6 secondes. Aucune erreur n'apparaît, le serveur n'est pas saturé en mémoire, et les disques sont sains. Décris la démarche complète de diagnostic et de correction : quels outils, quelles observations attendues, quel correctif, quelles vérifications après coup, et quelles mesures de routine pour éviter la récidive.

✔ Corrigé du TD 2

Démarche : 1) Reproduire et mesurer la requête exacte (celle de l'application) et son temps. 2) Lire son plan avec EXPLAIN ANALYZE : l'observation attendue est un balayage séquentiel (Seq Scan) sur la table des ouvrages avec un filtre sur la colonne auteur — le moteur lit 900 000 lignes pour en rendre une poignée. Cela explique la proportionnalité : à 20 000 lignes le balayage coûtait déjà son prix, mais il restait invisible. 3) Correctif : créer l'index manquant — CREATE INDEX sur la colonne auteur (éventuellement composite auteur + titre si le tri est demandé). 4) Vérification : nouvel EXPLAIN (le plan doit passer en Index Scan), temps de réponse mesuré à nouveau (attendu : quelques millisecondes), et contrôle que les écritures (catalogage) ne sont pas dégradées de façon notable. 5) Routine : activer le journal des requêtes lentes avec un seuil (par exemple 1 seconde), garder les statistiques à jour (ANALYZE / autovacuum), revoir chaque mois les requêtes les plus coûteuses et les index inutilisés. La leçon générale : les performances se dégradent avec le volume, pas avec le temps — un plan lu aujourd'hui vaut mieux qu'une devinette le jour de la plainte.

Travaux pratiques (TP)

TP 1 — Mettre en exploitation une base de bibliothèque

Sur une machine Linux (virtuelle si besoin) : 1) installe PostgreSQL, fixe UTF-8 et crée la base biblio avec les tables livres et emprunts ; 2) crée les trois rôles de la leçon 5 (application, lecture, administration nominative) et prouve qu'aucun n'est superutilisateur ; 3) écris et planifie par cron une sauvegarde complète nocturne horodatée, avec journal ; 4) insère quelques emprunts, lance la sauvegarde, puis simule le désastre : supprime la base, restaure-la et montre que les emprunts sont revenus ; 5) crée un index sur la colonne la plus filtrée et compare les EXPLAIN avant/après ; 6) rédige une fiche d'exploitation d'une page : versions, paramètres modifiés, calendrier des tâches, procédure de restauration.

✔ Attendus du TP 1
Attendus : l'installation se termine par une connexion psql réussie et pg_reload_conf sans erreur. La commande \du montre les trois rôles sans attribut « Superutilisateur », et un test de connexion avec le rôle lecture échoue proprement sur un INSERT (permission refusée) — c'est la preuve du moindre privilège. La ligne cron produit chaque nuit un fichier daté de taille non nulle et une ligne de journal. L'étape 4 est le cœur du TP : la restauration doit être démontrée, chronométrée, et le comptage des emprunts avant suppression et après restauration doit être identique. L'étape 5 reprend l'exemple réel de la leçon 4 (SCAN devenu SEARCH après index, ou l'équivalent PostgreSQL Seq Scan devenu Index Scan). La fiche d'exploitation, enfin, tient en une page mais contient l'essentiel : qui appeler, quoi restaurer, dans quel ordre. Un TP réussi n'est pas celui qui « marche » : c'est celui dont la procédure de restauration est écrite et prouvée.

Exercices corrigés supplémentaires

Exercice 1

Une association sauvegarde ainsi : complète le 1er de chaque mois, incrémentales chaque nuit. Le 20 au matin, on doit restaurer l'état du 19 au soir. 1) Combien de fichiers faut-il appliquer, dans quel ordre ? 2) Réponds à la même question si les sauvegardes nocturnes étaient différentielles. 3) L'incrémentale du 12 est corrompue : quelle est la conséquence dans chaque schéma ? 4) Quel est le RPO de cette organisation si un crash survient le 20 à 15 h, sans archivage de journaux ?

✔ Corrigé
1) Incrémentales : la complète du 1er + les 18 incrémentales des nuits du 2 au 19, soit 19 fichiers appliqués dans l'ordre chronologique strict. 2) Différentielles : 2 fichiers — la complète du 1er et la différentielle du 19 au soir, qui cumule tous les changements depuis le 1er. 3) Incrémentale du 12 corrompue : impossible de dépasser le 11 au soir — toute la chaîne postérieure est inutilisable, on perd 8 jours de recul. En différentiel, la corruption de celle du 12 est sans effet sur la restauration du 19 (seule la différentielle du 19 compte) ; en revanche, un souci sur la complète du 1er est fatal aux deux schémas. 4) Le dernier état sain est celui du 19 au soir : le RPO réel va jusqu'à environ 24 h de données perdues (les écritures du 20 jusqu'à 15 h), malgré des sauvegardes « quotidiennes » — sans journaux archivés, la fréquence nocturne fixe la perte maximale.

Exercice 2

Un script d'audit découvre ces deux lignes dans une base de production : GRANT ALL PRIVILEGES ON DATABASE boutique TO app_web ; et CREATE ROLE stagiaire SUPERUSER LOGIN PASSWORD 'ete2026'. Identifie tout ce qui cloche (au moins quatre problèmes), puis écris les commandes corrigées pour un usage normal : l'application gère les tables de la base boutique, le stagiaire consulte en lecture seule sous un compte nominatif.

✔ Corrigé
Problèmes : (1) ALL PRIVILEGES pour un compte applicatif viole le moindre privilège (il inclut des droits d'administration inutiles) ; (2) SUPERUSER pour un stagiaire est le droit maximal sur tout le serveur, sans aucune justification ; (3) le mot de passe 'ete2026' est faible, devinable, saisonnier — et il a voyagé en clair dans un script ; (4) le compte « stagiaire » n'est pas nominatif : impossible de tracer les actions d'une personne ; (5) aucun hôte n'est restreint dans pg_hba.conf par défaut. Version corrigée : CREATE ROLE app_web LOGIN PASSWORD 'MotDePasseFortFictif!1' ; GRANT CONNECT ON DATABASE boutique TO app_web ; GRANT USAGE ON SCHEMA public TO app_web ; GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_web ; et pour le stagiaire, un compte à son nom : CREATE ROLE a_diallo LOGIN PASSWORD 'AutreFictifFort!2' ; GRANT CONNECT ON DATABASE boutique TO a_diallo ; GRANT USAGE ON SCHEMA public TO a_diallo ; GRANT SELECT ON ALL TABLES IN SCHEMA public TO a_diallo ; Puis on supprime le compte stagiaire SUPERUSER et l'on fait tourner (rotation) tout mot de passe ayant circulé.

Exercice 3

Une base grandit de 2 Go par mois ; son disque dédié de 200 Go est déjà occupé à 120 Go (dont 60 Go pour la base elle-même). Les sauvegardes complètes hebdomadaires (10 Go compressées chacune, rétention 4) sont stockées sur ce même disque, le reste de l'occupation étant le système et ses journaux. 1) Dans combien de mois environ le disque sera-t-il plein si rien ne change ? 2) Cite deux décisions d'exploitation qui corrigent durablement la trajectoire. 3) Quel seuil d'alerte proposerais-tu à la supervision ?

✔ Corrigé
1) Les 4 complètes conservées occupent 4 × 10 = 40 Go en régime permanent (chaque semaine, une entre et une sort). Une complète compressée vaut environ 1/6 de la base (10 Go pour 60 Go). La base croît de 2 Go par mois, donc l'ensemble base + sauvegardes croît d'environ 2 + 4 × (2/6) ≈ 3,3 Go par mois. Avec 200 − 120 = 80 Go libres, le disque se remplit en 80 / 3,3 ≈ 24 mois — à condition que la purge fonctionne réellement et qu'aucune rétention exceptionnelle ne s'ajoute. Mais le vrai problème est ailleurs : les sauvegardes vivent sur le même disque que les données, ce qui viole la règle 3-2-1 — la panne de ce disque détruirait l'original et ses copies le même jour. 2) Deux décisions durables : externaliser les sauvegardes vers un autre support, avec une copie hors site ; et automatiser la purge contrôlée des rétentions (leçon 6), en séparant à terme le volume des données et celui des sauvegardes. 3) Alerte à 75-80 % d'occupation, accompagnée d'une projection de tendance (« plein estimé dans X jours ») — pas à 95 %, quand il est déjà trop tard.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Lors d'une restauration différentielle, quels fichiers faut-il appliquer ?

A) La dernière différentielle seule

B) La dernière complète, puis la dernière différentielle

C) La complète et toutes les différentielles depuis

2. Dans une sauvegarde incrémentale, chaque fichier contient…

A) Toutes les données de la base

B) Les changements depuis la sauvegarde complète uniquement

C) Les changements depuis la dernière sauvegarde, complète ou incrémentale

3. Le RPO (Recovery Point Objective) mesure…

A) La quantité maximale de données que l'on accepte de perdre, exprimée en temps

B) La durée maximale d'interruption du service

C) Le temps moyen de réparation d'un disque

4. Un plan d'exécution affiche un balayage séquentiel complet d'une table de 900 000 lignes pour un filtre très sélectif. Le diagnostic le plus probable est…

A) Le disque est défaillant

B) La requête contient une faute de syntaxe

C) Il manque un index sur la colonne filtrée

5. Le principe du moindre privilège impose que le compte de l'application soit…

A) Superutilisateur, pour ne jamais être bloqué

B) Limité aux opérations et aux tables dont l'application a besoin

C) Partagé entre toutes les applications du serveur

6. En réplication synchrone, le primaire valide une transaction…

A) Immédiatement, sans attendre personne

B) Uniquement lors de la sauvegarde de la nuit

C) Seulement après confirmation d'au moins un secondaire

7. Une réplique en flux (streaming) protège contre…

A) La panne matérielle du serveur primaire

B) Un DELETE massif exécuté par erreur

C) La suppression accidentelle d'une table

8. Que désigne, dans une ligne cron « 30 4 * * 1-5 », l'expression « 1-5 » ?

A) Les mois de janvier à mai

B) Les jours de semaine, du lundi au vendredi

C) Les jours du mois de 1 à 5

9. Pour restaurer une base « à 10 h 04 », juste avant une fausse manipulation de 10 h 05, il faut…

A) Une sauvegarde différentielle de 10 h 00

B) Un index sur la colonne de la date

C) Une sauvegarde de base et l'archivage des journaux de transactions (PITR)

10. Quel contrôle relève de la routine quotidienne du DBA ?

A) La migration vers une nouvelle version majeure

B) Vérifier le succès des sauvegardes de la nuit et le retard de réplication

C) La revue annuelle des contrats de maintenance

Corrigés du QCM

1. Réponse B — La différentielle contient tous les changements depuis la dernière complète : deux fichiers suffisent, la complète puis la dernière différentielle.

2. Réponse C — C'est la définition de l'incrémentale — et la raison pour laquelle la restauration doit rejouer toute la chaîne dans l'ordre, sans maillon manquant.

3. Réponse A — RPO = jusqu'à quel point dans le temps on peut revenir ; c'est le RTO qui mesure la durée d'interruption acceptable.

4. Réponse C — Seq Scan / SCAN sur un filtre qui ne retient que quelques lignes : le moteur lit tout, faute d'index — le correctif classique consiste à créer cet index puis à re-mesurer.

5. Réponse B — Moindre privilège : SELECT/INSERT/UPDATE/DELETE sur le schéma métier, jamais superutilisateur — une faille applicative ne doit pas offrir tout le serveur.

6. Réponse C — C'est le compromis de la synchronie : aucune perte de transaction confirmée, contre une latence d'écriture plus élevée et un secondaire qui doit rester joignable.

7. Réponse A — La réplication copie aussi les erreurs, instantanément. Contre les fautes logiques, seules les sauvegardes (et le PITR) protègent ; contre la panne matérielle, la réplique prend le relais.

8. Réponse B — Les champs cron sont : minute, heure, jour du mois, mois, jour de semaine (0 = dimanche) — « 1-5 » en cinquième position désigne donc lundi à vendredi, ici à 4 h 30.

9. Réponse C — Le PITR (Point-In-Time Recovery) rejoue les journaux (WAL ou binary logs) après la sauvegarde de base et s'arrête à l'instant choisi : c'est l'outil du retour en arrière précis.

10. Réponse B — Quotidien : sauvegardes de la nuit, erreurs nouvelles des journaux, espace disque, retard du secondaire. Le reste est hebdomadaire, mensuel ou exceptionnel.

Résumé — les notions importantes

• Installer un SGBD serveur, c'est décider : emplacement des données, encodage UTF-8, accès réseau au plus juste (pg_hba.conf), mémoire et connexions dimensionnées ensemble — puis documenter.

• Complète, différentielle (depuis la dernière complète), incrémentale (depuis la dernière sauvegarde) : la famille choisie détermine combien de fichiers une restauration exige.

• RPO et RTO dimensionnent la stratégie : fréquence et journaux pour la perte de données, moyens de restauration et bascule pour la durée d'interruption — et des

tests de restauration pour le prouver.

• La réplication (flux de journaux) sert la lecture répartie et la haute disponibilité, en mode asynchrone (rapide, perte possible) ou synchrone (zéro perte confirmée, plus lent) ; elle ne remplace jamais une sauvegarde.

• EXPLAIN montre le plan réel d'une requête : un balayage complet sur une grosse table filtrée signale un index manquant ; l'index créé se paie en espace et en écritures plus coûteuses.

• Sécurité : rôles au moindre privilège (jamais de superutilisateur applicatif), mots de passe à empreinte, connexions chiffrées, droits par défaut retirés, audit relu.

• Automatiser (cron) sauvegardes, purges et contrôles — en supervisant l'automate lui-même ; superviser chaque jour service, espace, lenteurs et protections, avec seuils d'alerte et procédures d'incident écrites.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Ouvrir pgAdmin et se connecter à la base sur Mac | SQL sur PostgreSQL, leçon 0b

Chaîne : Diallo Géo · Langue : Français · Niveau : Intermédiaire

Premiers pas d'administration concrète : ouvrir pgAdmin, se connecter au serveur PostgreSQL et retrouver ses bases. Un bon complément visuel à la leçon 1 avant de travailler en ligne de commande avec psql.

https://www.youtube.com/watch?v=-ywvAF2Vku4

Leçon 2 — Leçon 1_Partie_4_Eviter les pertes de données - Sauvegarde - Mon Prof d'Info

Chaîne : Mon Prof D'Info · Langue : Français · Niveau : Intermédiaire

Pourquoi sauvegarder, quelles sont les causes de perte de données, et comment bâtir une stratégie : types de sauvegardes, multiplication des supports et règle 3-2-1. À regarder en écho direct à la leçon 2 de ce cours.

https://www.youtube.com/watch?v=EDYgmYD6HPk

Leçon 4 — SQL avec PostgreSQL : le cours complet (2 h 29, 34 chapitres, 99 exercices)

Chaîne : lioncode · Langue : Français · Niveau : Intermédiaire à avancé

Un cours complet en français dont les chapitres « Les index », « L'optimiseur, les statistiques et VACUUM », « Rôles, droits et sécurité » et « Exploiter un serveur PostgreSQL » prolongent directement les leçons 4, 5 et 7 de ce cours.

https://www.youtube.com/watch?v=rpVy7sIM9BY

Leçon 6 — Cet outil open source change la vie pour la sauvegarde des bases de données

Chaîne : IT-Connect - Florian · Langue : Français · Niveau : Intermédiaire

Présentation d'un outil libre qui centralise sauvegarde et restauration de bases PostgreSQL, MySQL et MariaDB : une illustration concrète de l'automatisation traitée dans la leçon 6, au-delà du simple script cron.

https://www.youtube.com/watch?v=nWsnwDjyCg8

Sources et références

• Documentation officielle PostgreSQL (postgresql.org/docs) : chapitres « Server Administration », sauvegarde et restauration (pg_dump, pg_basebackup, PITR), réplication en flux, EXPLAIN — la référence de ce cours.

• Documentation officielle MySQL (dev.mysql.com/doc) : mysqldump, binary logs et réplication, InnoDB Cluster — pour appliquer les mêmes concepts côté MySQL.

• Documentation SQLite (sqlite.org) : EXPLAIN QUERY PLAN et CREATE INDEX — le terrain des expérimentations réelles de la leçon 4.

• ANSSI — Guide d'hygiène informatique : les règles relatives à la sauvegarde des données et à la gestion des comptes d'administration.

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !